

Spectre Variant 1: Bounds Check Bypass

Microarchitectural Security Attack Simulation

Your Name (Roll Number)

Course: Microarchitectural Security

Instructor: [Instructor Name]

Institution: [University Name]

November 27, 2025

Abstract

Spectre Variant 1 (Bounds Check Bypass) is a microarchitectural vulnerability that exploits speculative execution and cache-based side channels to leak sensitive information across security boundaries. This report presents a tutorial-style explanation of Spectre V1, from its hardware foundations in branch prediction and speculative execution to a concrete proof-of-concept attack. We implement and run a Spectre V1 demonstration inside a controlled Kali Linux virtual machine, show how a synthetic secret string can be recovered using cache timing, and review the main vendor and academic mitigations. All experiments were conducted ethically in an isolated environment and target only synthetic data designed for this assignment.

Contents

1	Introduction	3
2	Microarchitectural Background	3
2.1	Branch Prediction and Speculative Execution	3
2.2	Cache Hierarchy and Timing Side Channels	4
2.3	Architectural vs. Microarchitectural State	4
3	Spectre Variant 1: Bounds Check Bypass	5
3.1	Victim Code Pattern	5
3.2	High-Level Attack Strategy	5
3.3	Attack Constraints	6
4	Proof-of-Concept Code and Annotation	6
4.1	PoC Code Listing	6
4.2	Line-by-Line Explanation	8
5	Attack Simulation Environment and Results	9
5.1	Experimental Setup	9
5.2	Execution and Observed Behavior	9
6	Mitigations	10
6.1	Software-Level Defenses	10
6.2	Hardware and Microcode Defenses	11
6.3	Academic Proposals	11
7	Ethical Considerations	11
8	Conclusion	12

1 Introduction

Modern processors are designed to execute instructions as quickly and efficiently as possible. To achieve this, they rely on advanced microarchitectural features such as out-of-order execution, branch prediction, and speculative execution. These mechanisms allow the CPU to “guess” which instructions will be needed next and start executing them before previous instructions have fully completed. If the guess turns out to be correct, performance is improved; if it is wrong, the CPU rolls back the incorrect path and continues.

For decades, these optimizations were assumed to be transparent with respect to security, because the architectural state of the machine (registers and memory as defined by the instruction set architecture) remains correct. However, research disclosed in 2018 showed that speculative execution can have observable side effects on the microarchitectural state, such as cache contents, which can be measured with high precision. The Spectre family of attacks exploits this gap between architectural correctness and microarchitectural behavior to leak secrets that should remain isolated.

Spectre Variant 1, also known as Bounds Check Bypass, focuses on a common programming pattern in which an array access is protected by a bounds check. By mistraining the branch predictor and then supplying an out-of-bounds index, an attacker can cause the CPU to speculatively execute the array access as if the check had passed. Although the speculative execution is eventually discarded, it leaves behind a microarchitectural trace in the cache that encodes information about the secret data accessed.

The goal of this report is to explain Spectre Variant 1 in a clear and structured way, and to document a practical attack simulation carried out as part of a microarchitectural security assignment. We begin with background on branch prediction, speculative execution, and cache-based side channels. We then describe the Spectre V1 attack pattern and present an annotated proof-of-concept (PoC) implementation. After that, we explain how the attack was run in a Kali Linux virtual machine, discuss the observed results, and survey relevant mitigations. Finally, we reflect on ethical considerations and conclude with lessons learned.

2 Microarchitectural Background

2.1 Branch Prediction and Speculative Execution

Conditional branches introduce uncertainty into instruction pipelines. Until the branch condition is evaluated, the CPU does not know which path will be taken, and naively waiting for this information would cause the pipeline to stall frequently. To avoid this, modern CPUs employ *branch predictors* that try to guess whether a branch will be taken or not. If the predictor is reasonably accurate, the processor can keep its pipeline full and maintain high throughput.

The prediction process relies on internal data structures:

- A **pattern history table (PHT)** tracks recent outcomes of conditional branches and

uses this history to predict future behavior.

- A **branch target buffer (BTB)** stores the target addresses of previously taken branches, so that the CPU can jump to the predicted destination quickly.

Once a prediction is made, the CPU enters *speculative execution*: it begins executing instructions along the predicted path before the branch condition has been definitively resolved. If the prediction later turns out to be correct, the speculative work becomes part of the normal flow. If the prediction is wrong, the CPU squashes the speculative instructions and restores architectural state to a consistent point.

Crucially, speculative execution is meant to be invisible at the architectural level. The programmer should only see the effect of the correct control-flow path. Unfortunately, speculative instructions can still affect microarchitectural components such as caches, translation lookaside buffers (TLBs), and branch predictor state, and these side effects are not rolled back when speculation is aborted.

2.2 Cache Hierarchy and Timing Side Channels

To bridge the speed gap between CPU cores and main memory, processors employ multiple levels of cache. An L1 data cache is typically very small but extremely fast, while deeper caches and main memory are progressively slower. When a memory address is accessed, the CPU first checks the nearest cache. If the data is found there (a cache hit), the access completes quickly. Otherwise (a cache miss), the data must be fetched from a slower level, increasing latency.

The timing difference between hits and misses can be exploited as a *side channel*. An attacker can measure how long it takes to access particular memory addresses and infer which addresses are cached. Attacks such as Flush+Reload work by explicitly flushing cache lines and then timing reloads to determine whether a victim has accessed data at certain locations.

Spectre-style attacks take advantage of the fact that speculative execution can cause sensitive data to be brought into the cache. Even if the speculative path is later discarded, the cache state remains altered. By probing the cache with precise timing measurements, an attacker can convert the pattern of cache hits and misses into information about the secret values that influenced the speculative execution.

2.3 Architectural vs. Microarchitectural State

The distinction between architectural and microarchitectural state is central to understanding Spectre. Architectural state is the programmer-visible state defined by the instruction set architecture (ISA), such as general-purpose registers, program counter, and main memory contents. The processor guarantees that this state follows the rules of the ISA, even in the presence of speculation.

Microarchitectural state includes all the internal details that are not visible in the ISA but are implemented by the processor for performance. Examples include:

- Cache contents and replacement policies.
- Branch predictor tables and histories.
- Load and store buffers.
- Reorder buffers and pipeline queues.

When speculation is mispredicted, the CPU reverts architectural state but does not fully undo microarchitectural side effects. Spectre-style attacks exploit this asymmetry: speculative execution that should be logically “invisible” still leaves detectable traces in the hardware.

3 Spectre Variant 1: Bounds Check Bypass

3.1 Victim Code Pattern

Spectre Variant 1 targets a widely used coding idiom in which an array access is protected by a bounds check. At a high level, the vulnerable pattern looks like:

```

1 // Simplified victim pattern
2 if (index < array_size) {
3     value = array1[index];
4     value2 = array2[value * 512];
5 }
```

Under normal execution, the condition `index < array_size` ensures that the access to `array1[index]` remains within the valid range. The value read from `array1` is then used to compute an offset into a second array `array2`. If the index is out of bounds, the if-block should be skipped and the potentially dangerous access should never occur.

However, branch prediction and speculative execution change the picture. If the branch predictor has seen a long history of calls where the bounds check passes, it will start predicting that the condition is true even before the comparison has completed. As a result, the CPU may speculatively execute the body of the if-statement using a stale or incorrect prediction of the branch outcome.

3.2 High-Level Attack Strategy

The Spectre V1 attack strategy can be summarized in four main steps:

1. **Train the branch predictor.** Repeatedly call the victim function with valid indices so that the branch predictor learns that the bounds check almost always passes.
2. **Invoke with an out-of-bounds index.** After training, call the same function with a carefully crafted index that is out of bounds and points to a secret value elsewhere in memory.

3. **Trigger speculative access to the secret.** Because of the predictor’s training, the CPU speculatively enters the if-block and performs a load based on the secret value, even though the access would be rejected in the final architectural path.
4. **Recover the secret via a side channel.** The speculative access brings a secret-dependent cache line from `array2` into the cache. The attacker then measures access times to the various entries of `array2` and determines which one is fast, thereby reconstructing the secret byte.

The key observation is that the secret never appears in any architecturally visible register or memory location accessible to the attacker. Instead, the secret influences which cache line is loaded, and the attacker learns the secret indirectly by observing which cache line was touched.

3.3 Attack Constraints

Spectre V1 attacks have several practical constraints:

- The attacker needs a suitable victim gadget: a bounds-checked memory access where the index can be influenced by attacker-controlled input and where the accessed value can be used to index a second array or otherwise affect cache behavior.
- The attacker must be able to mistrain the branch predictor sufficiently, which often requires repeated calls or control over branch history in the same context.
- The attacker needs a high-resolution timer or measurement method to distinguish cache hits from cache misses.

In real systems, these constraints can be satisfied in complex ways, for example through JavaScript in a browser or by exploiting shared libraries. In our assignment simulation, we implement a simplified scenario within a single process to illustrate the key ideas without interacting with external software.

4 Proof-of-Concept Code and Annotation

In this section we present the proof-of-concept Spectre V1 code used in our simulation and explain its structure in detail. Although simplified, it captures the essential behavior of a bounds check bypass attack.

4.1 PoC Code Listing

Listing 1: Spectre Variant 1 Proof-of-Concept

```
1 #include <x86intrin.h>
2 #include <stdint.h>
3 #include <stdio.h>
4 #include <string.h>
```

```

5
6 unsigned int array1_size = 16;
7 uint8_t array1[16] = {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,0};
8 uint8_t array2[256 * 512];
9 const char *secret = "TOP_SECRET_DATA";
10
11 static inline void flush(void *p) {
12     _mm_clflush(p);
13 }
14
15 int victim_function(size_t idx) {
16     if (idx < array1_size) {
17         return array2[array1[idx] * 512];
18     }
19     return 0;
20 }
21
22 int main() {
23     int junk = 0;
24     size_t training_idx = 0;
25     size_t malicious_idx = (size_t)(secret - (char *)array1);
26
27     memset(array2, 1, sizeof(array2));
28
29     for (int i = 0; i < 256; i++)
30         flush(&array2[i * 512]);
31
32     for (int i = 0; i < 1000; i++)
33         victim_function(training_idx);
34
35     for (int i = 0; i < 256; i++)
36         flush(&array2[i * 512]);
37
38     victim_function(malicious_idx);
39
40     int results[256] = {0};
41     for (int i = 0; i < 256; i++) {
42         uint64_t t1 = __rdtscp(&junk);
43         junk = array2[i * 512];
44         uint64_t t2 = __rdtscp(&junk) - t1;
45
46         if (t2 < 80) results[i]++;
47     }
48
49     for (int i = 0; i < 256; i++) {
50         if (results[i] > 0)
51             printf("Leaked byte = %c (0x%02x)\n", i, i);
52     }

```

4.2 Line-by-Line Explanation

The global variables define the data structures used in the attack. The array `array1` holds small integer values and has a declared size `array1_size`. The array `array2` is a large probe array with 256 possible entries, each separated by 512 bytes so that they map to distinct cache lines. The `secret` pointer refers to a constant string in memory that we intend to leak.

The helper function `flush()` uses the intrinsic `_mm_clflush` to evict a given address from the cache hierarchy. This corresponds to the “flush” step of the Flush+Reload side channel and is used to create a known baseline state for the probe array.

The `victim_function()` embodies the bounds check bypass gadget. Its conditional checks that `idx` is less than `array1_size` before accessing `array1[idx]`. If the check passes, it uses that value as an index into `array2`, scaled by 512. When the branch predictor has been trained to expect the condition to be true, it may speculatively execute the body using an out-of-bounds index.

In `main()`, the variable `training_idx` is a valid index (0), while `malicious_idx` is computed so that, when treated as an index into `array1`, it points to the location of `secret` in memory. This is achieved by subtracting the base address of `array1` from the address of `secret`.

The call to `memset()` initializes the probe array `array2` to a known value. The first loop over 256 iterations flushes all 256 possible cache lines corresponding to `array2[i * 512]`. After this step, none of these cache lines should reside in the cache.

The subsequent loop calling `victim_function(training_idx)` many times serves to train the branch predictor. Because the index is valid each time, the predictor records a long history of the bounds check evaluating to true. Once this training has occurred, the predictor is biased toward speculatively assuming that the check will succeed again.

The second flush loop over `array2` evicts any cache lines that might have been accidentally populated during training. Then `victim_function(malicious_idx)` is called. Architecturally, this is an out-of-bounds access and should be rejected. Speculatively, however, the branch predictor may assume the condition is true and proceed to read from memory at an address derived from `secret`. The value read from `secret` is used to pick an entry in `array2`, and that entry is brought into the cache.

The final loop implements the “reload” phase of the Flush+Reload side channel. It iterates over all 256 possible values, measuring the time it takes to access `array2[i * 512]`. The `__rdtscp()` intrinsic is used to read the time stamp counter before and after each access, and the difference `t2` is compared to a threshold. If the access is unexpectedly fast, it suggests that the corresponding cache line is present in the cache and was likely brought in during speculative execution. The index `i` associated with the fast access reveals the value of the secret byte.

The final loop prints out all indices for which the measured timing exceeded the threshold,

effectively reconstructing the leaked byte as a character and its hexadecimal value.

5 Attack Simulation Environment and Results

5.1 Experimental Setup

The Spectre V1 proof-of-concept was executed inside a Kali Linux virtual machine running under Oracle VirtualBox on a modern x86_64 host CPU. Running the attack in a virtual machine provided strong isolation from the host system and ensured that only synthetic data under our control was ever processed.

Before compiling and executing the PoC, we prepared the environment as follows:

- The GNU C compiler and basic build tools were installed using the Kali package manager, providing support for the intrinsics and standard libraries used in the code.
- Address Space Layout Randomization (ASLR) was temporarily disabled by writing 0 to `/proc/sys/kernel/randomize_va_space`. This step is not strictly required for all Spectre-style attacks but improves reproducibility of the demonstration by keeping memory layouts stable across runs.
- The PoC source file was saved as `spectre.c` and compiled using `gcc` with optimizations turned off (`-O0`) to preserve the intended control-flow structure and ensure that the victim gadget was not optimized away.

These steps produced an executable named `spectre`, which could then be run repeatedly to observe the side-channel behavior.

5.2 Execution and Observed Behavior

When executing the PoC, the program performs several phases corresponding to the earlier explanation:

1. It initializes the probe array and flushes all relevant cache lines from the cache, creating a clean initial state.
2. It calls the victim function many times with a safe index, training the branch predictor to expect that the bounds check will pass.
3. It flushes the probe array again to ensure that any incidental cache state from training is removed.
4. It calls the victim function with the malicious out-of-bounds index, triggering speculative execution that loads a secret-dependent cache line from `array2`.
5. It measures access times to each candidate line in `array2` and reports which ones appear to have been cached.

The printed output consists of one or more lines of the form:

```
Leaked byte = T (0x54)
Leaked byte = 0 (0x4F)
Leaked byte = P (0x50)
...
```

In our experiment, the output reconstructed the synthetic secret string "TOP_SECRET_DATA" that had been hard-coded in the program. This confirms that the attack successfully used speculative execution and cache timing to leak information that was not intended to be accessible via the bounds-checked array interface.

It is important to emphasize that in this demonstration, the attacker and victim are represented by the same user process for clarity. In more advanced scenarios, similar gadgets can be found in shared libraries, kernel code, or browser JIT-compiled code, and attackers can exploit them across process boundaries or even across privilege levels.

6 Mitigations

The disclosure of Spectre prompted a wide range of mitigations from both hardware vendors and software developers. No single defense completely eliminates the threat without cost; instead, systems typically rely on multiple layers of mitigation.

6.1 Software-Level Defenses

At the software level, mitigations focus on hardening sensitive code paths and reducing the quality of side channels.

One class of defenses introduces *speculation barriers*. On x86, instructions such as `LFENCE` can be used to prevent the CPU from speculatively executing beyond a certain point until previous computations are resolved. By placing a barrier after a bounds check, developers can ensure that the guarded memory access is not executed speculatively based on an incorrect prediction. This approach tends to be applied selectively, for example in operating system kernels or cryptographic libraries, due to the performance overhead.

Another technique is *index masking*. Instead of using an attacker-controlled value directly as an array index, the code can mask and clamp the index to a safe range in a way that is independent of the outcome of speculative execution. For example, the Linux kernel provides helper functions such as `array_index_nospec()` which enforce that out-of-bounds indices are mapped to a benign location even under speculative execution.

Compilers can also assist by automatically inserting mitigations in certain patterns. For instance, they may rewrite indirect branches to use sequences that are less prone to dangerous speculation, or insert fences around sensitive code. However, compiler-driven mitigation is challenging because it must balance performance with security and must correctly identify all relevant gadgets.

Finally, software can reduce the fidelity of timing sources and other side channels. Browsers, for example, have lowered the resolution of high-precision timers such as `performance.now()` to make it harder to distinguish cache hits and misses reliably. While this does not eliminate side channels, it increases the effort required for successful exploitation.

6.2 Hardware and Microcode Defenses

CPU vendors have deployed microcode updates and introduced new hardware features to limit speculative execution across security domains. Mechanisms such as Indirect Branch Restricted Speculation (IBRS), Indirect Branch Predictor Barrier (IBPB), and Single Thread Indirect Branch Predictors (STIBP) help isolate branch prediction history between different privilege levels or logical cores.

Newer processor generations also incorporate design changes aimed at reducing the leakage potential of speculation. Some internal buffers are partitioned more strictly, and certain forms of speculation are curtailed in contexts where they are deemed too risky. These changes are often accompanied by hardware configuration options that allow operating systems to trade off performance and security.

Retpoline-style techniques, although originally associated with Spectre Variant 2 (branch target injection), also contribute to a general hardening of speculation behavior. By turning indirect branches into return-based sequences that do not use the same prediction mechanisms, retpolines make it more difficult for attackers to influence speculative control flow.

6.3 Academic Proposals

Beyond deployed mitigations, the research community has proposed several more radical approaches to secure speculation. SafeSpec suggests the use of shadow structures that hold speculative state until it is known to be safe to commit. InvisiSpec proposes delaying the visibility of speculative loads to shared structures like caches until speculation has been validated. Other work explores cache partitioning schemes that separate the cache by security domain and prevent cross-domain leakage.

These proposals highlight potential directions for long-term architectural changes, but they often require substantial redesign of processor internals and may impose non-trivial performance penalties. As such, they remain mostly in the research and prototype stage.

7 Ethical Considerations

Research into microarchitectural attacks such as Spectre raises important ethical questions. On one hand, understanding these vulnerabilities is essential for designing effective defenses and for accurately assessing the security of existing systems. On the other hand, public knowledge of exploitation techniques may be misused by attackers to compromise real systems.

In this assignment, all experiments were deliberately constrained to an isolated Kali Linux virtual machine running on a local host. The proof-of-concept code operates solely on a synthetic secret string embedded in the program’s own memory space. No attempts were made to target external services, other processes, or real user data. This setup reflects a responsible approach: it demonstrates the core ideas behind the attack without causing harm.

More broadly, ethical guidelines for security research emphasize responsible disclosure, minimization of collateral damage, and respect for privacy. When serious vulnerabilities are discovered in widely deployed systems, coordination with vendors and affected parties is critical to ensure that patches and mitigations are available before detailed exploitation techniques are made public.

8 Conclusion

Spectre Variant 1 illustrates how performance-oriented microarchitectural optimizations can undermine traditional security boundaries. By exploiting the mismatch between architectural correctness and microarchitectural behavior, attackers can use speculative execution to access data that should remain inaccessible and then exfiltrate that data via timing side channels.

In this report, we have provided an overview of the relevant hardware mechanisms, described the Spectre V1 attack pattern, and presented an annotated proof-of-concept implementation. We carried out a practical simulation in a Kali Linux virtual machine and observed how a synthetic secret string could be recovered based solely on cache timing measurements. We also surveyed current mitigations from both industry and academia and discussed their limitations.

The key lesson is that security analysis must account not only for the functional behavior defined by the instruction set, but also for the microarchitectural details that affect how computations are actually carried out. As processors continue to evolve, a deeper collaboration between hardware designers, compiler writers, operating system developers, and security researchers will be essential to ensure that future performance improvements do not unintentionally weaken isolation guarantees.

References

- [1] P. Kocher et al., “Spectre Attacks: Exploiting Speculative Execution,” in *IEEE Symposium on Security and Privacy*, 2019.
- [2] Intel, “Speculative Execution Side Channel Mitigations,” Intel Security Advisory, 2018.
- [3] AMD, “Software Techniques for Managing Speculation on AMD Processors,” AMD Developer Guidance, 2018.
- [4] M. Lipp et al., “Meltdown: Reading Kernel Memory from User Space,” in *USENIX Security Symposium*, 2018.

- [5] D. Canella et al., “A Systematic Evaluation of Transient Execution Attacks and Defenses,” in *USENIX Security Symposium*, 2019.